

TRABAJO PRÁCTICO N° 2

Seminario de Práctica de Informática

Sistema de Gestión Integral para una Clínica de Salud

Alumna: Antonella Diaz

DNI: 28.910.424

Comisión: VINFOR12606

Repositorio GitHub: <https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

Índice

1. Etapa de Análisis
2. Etapa de Diseño
3. Diagramas de Secuencia
4. Etapa de Implementación
5. Etapa de Pruebas
6. Definición de Base de Datos
7. Diagrama Entidad-Relación
8. Creación de Tablas MySQL
9. Inserción, Consulta y Borrado de Registros
10. Presentación de Consultas SQL
11. Definiciones de Comunicación

1. Etapa de Análisis

1.1 Recolección de Información

Para comprender el funcionamiento de la clínica y relevar sus necesidades reales, se realizaron entrevistas con el personal médico, administrativo y de recepción. Este proceso permitió identificar las áreas críticas del sistema: la gestión de turnos, el manejo de historias clínicas, el control del inventario y la facturación. Se analizaron los procesos actuales —en su mayoría manuales o dispersos en hojas de cálculo— y se revisó la documentación existente para detectar inconsistencias y oportunidades de mejora.

1.2 Requerimientos del Sistema

A partir de la información recolectada, se identificaron los siguientes requerimientos funcionales que el sistema debe satisfacer:

- **Gestión de Citas:** Programación, modificación y cancelación de turnos médicos.
- **Historias Clínicas Electrónicas:** Creación, consulta y actualización de historias clínicas con acceso seguro.
- **Gestión de Inventarios:** Registro de productos, control de stock y alertas por vencimiento.
- **Facturación y Pagos:** Generación de facturas asociadas a pacientes y registro del estado de pago.
- **Reportes de Gestión:** Generación de reportes estadísticos sobre pacientes, turnos e ingresos.

1.3 Casos de Uso Principales

Se identificaron dos casos de uso centrales que definen las interacciones más importantes entre los actores y el sistema:

Caso de Uso 1 – Programación de Citas

Nombre	Programación de Citas
Actor Principal	Recepcionista / Paciente
Descripción	El sistema permite registrar un turno médico seleccionando paciente, médico, fecha y hora.
Precondición	El paciente debe estar registrado en el sistema y el médico debe tener disponibilidad.
Postcondición	La cita queda registrada en el sistema y disponible para consulta.
Flujo Principal	1. La recepcionista ingresa al módulo de citas. 2. Selecciona el paciente y el médico disponible. 3. Elige la fecha y hora del turno. 4. El sistema guarda el turno con estado "Pendiente".

	5. Se confirma el registro al usuario.
--	--

Caso de Uso 2 – Gestión de Historias Clínicas

Nombre	Gestión de Historias Clínicas
Actor Principal	Médico
Descripción	El médico registra o actualiza la historia clínica de un paciente tras una consulta.
Precondición	El paciente debe tener una cita confirmada con el médico.
Postcondición	La historia clínica queda registrada en la base de datos de forma segura.
Flujo Principal	1. El médico accede al sistema con sus credenciales. 2. Selecciona el paciente a atender. 3. Visualiza la historia clínica existente. 4. Registra el diagnóstico y el tratamiento indicado. 5. El sistema guarda la nueva entrada en la historia clínica.

2. Etapa de Diseño

2.1 Arquitectura del Sistema

El sistema fue diseñado con una arquitectura por capas que separa claramente las responsabilidades de cada componente. La capa de presentación, implementada en Java, interactúa con el usuario a través de un menú de consola. La capa de lógica de negocio procesa las operaciones mediante clases especializadas por entidad. La capa de acceso a datos utiliza el patrón DAO (Data Access Object), que encapsula todas las operaciones con la base de datos y desacopla la lógica de negocio del motor de persistencia. Finalmente, la capa de datos está implementada en MySQL, con tablas normalizadas y relaciones mediante claves foráneas.

2.2 Modelo de Datos Relacional

El modelo de datos fue diseñado para representar todas las entidades del dominio de la clínica y sus relaciones. Se definieron 8 tablas: Pacientes, Médicos, Citas, Historias_Clinicas, Inventarios, Facturas, Proveedores y Reportes. Las relaciones entre ellas se establecen mediante claves foráneas que garantizan la integridad referencial de los datos.

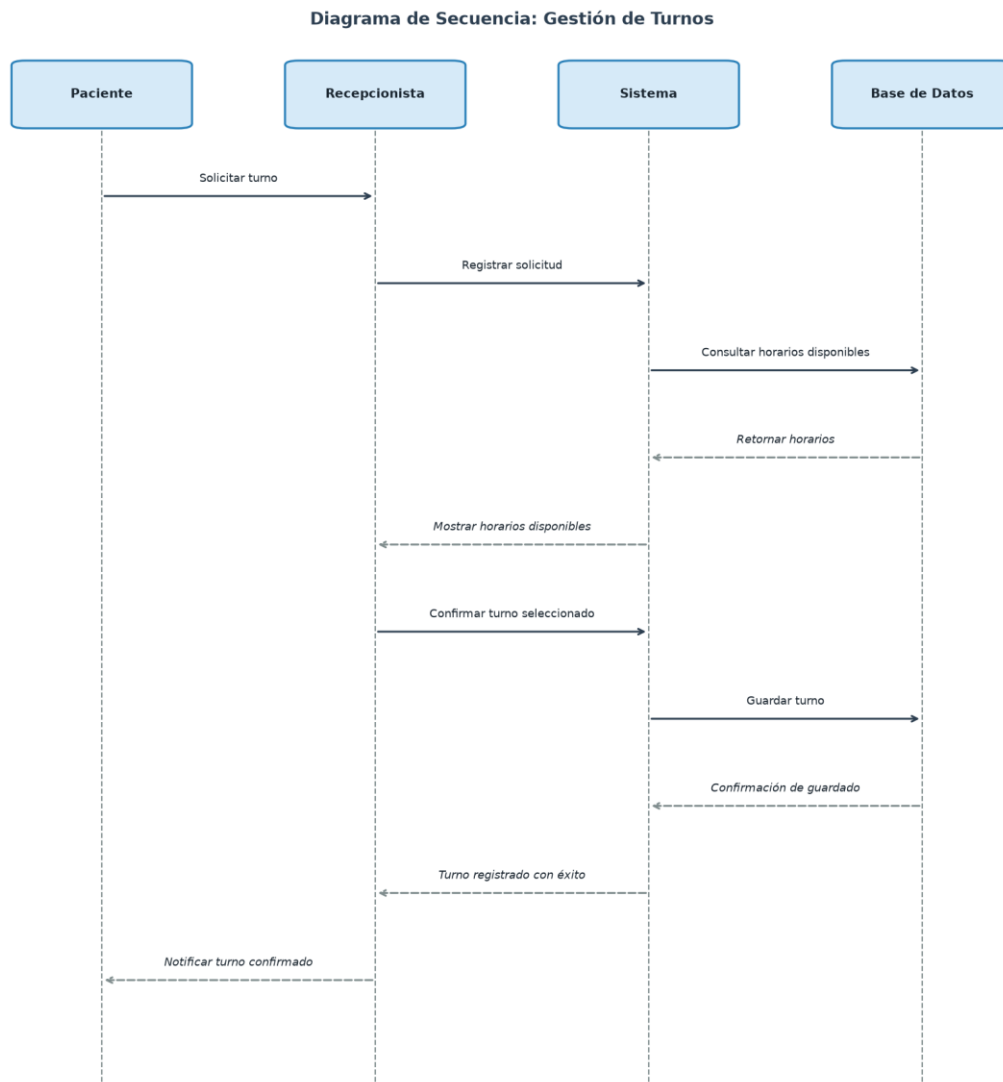
- **Pacientes:** id (PK), nombre, apellido, fecha_nacimiento, dirección, teléfono, correo_electronico
- **Médicos:** id (PK), nombre, apellido, especialidad, teléfono, correo_electronico
- **Citas:** id (PK), paciente_id (FK), medico_id (FK), fecha_hora, estado
- **Historias_Clinicas:** id (PK), paciente_id (FK), medico_id (FK), fecha, detalles
- **Inventarios:** id (PK), producto, cantidad, fecha_vencimiento, proveedor_id (FK)
- **Facturas:** id (PK), paciente_id (FK), fecha, total, estado
- **Proveedores:** id (PK), nombre, contacto
- **Reportes:** id (PK), tipo_reporte, fecha_creacion, detalles

3. Diagramas de Secuencia

Los diagramas de secuencia describen cómo interactúan los actores con el sistema a lo largo del tiempo para completar un proceso específico. Las flechas continuas representan mensajes de invocación y las flechas discontinuas representan respuestas o retornos.

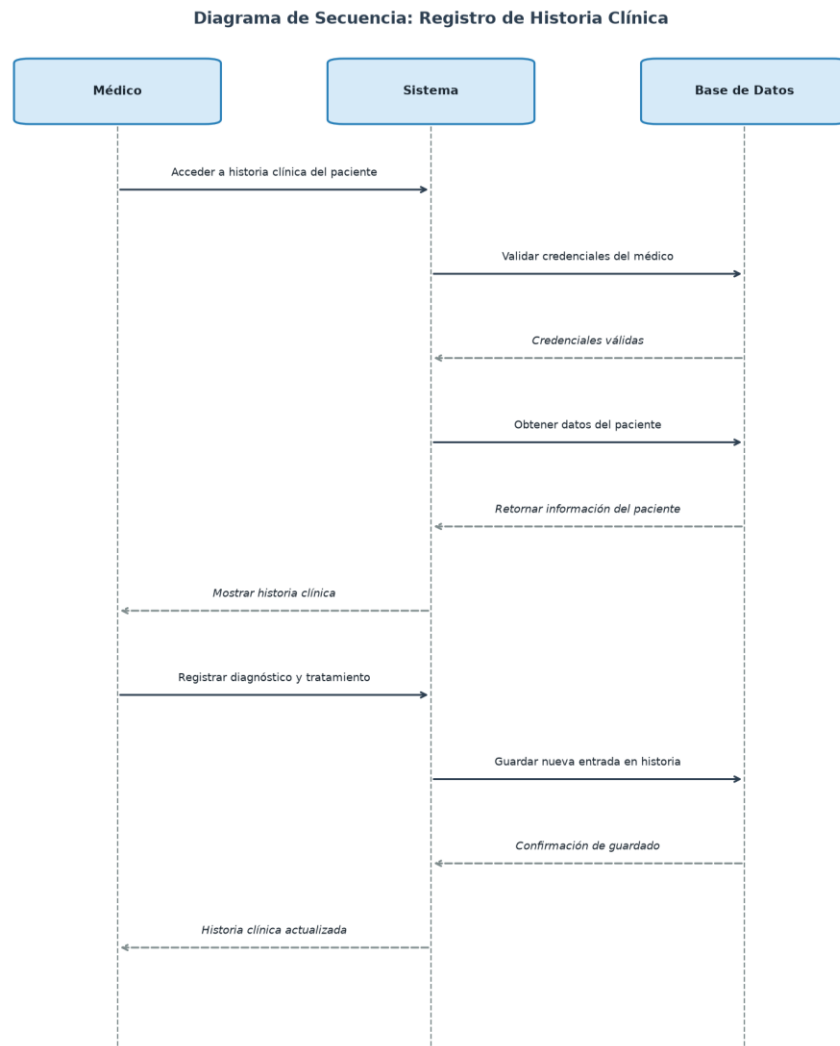
3.1 Gestión de Turnos

El siguiente diagrama muestra el flujo completo para programar un turno médico. El paciente solicita el turno a través de la recepcionista, quien lo registra en el sistema. El sistema consulta los horarios disponibles en la base de datos y, una vez confirmado, guarda el turno y notifica al paciente.



3.2 Registro de Historia Clínica

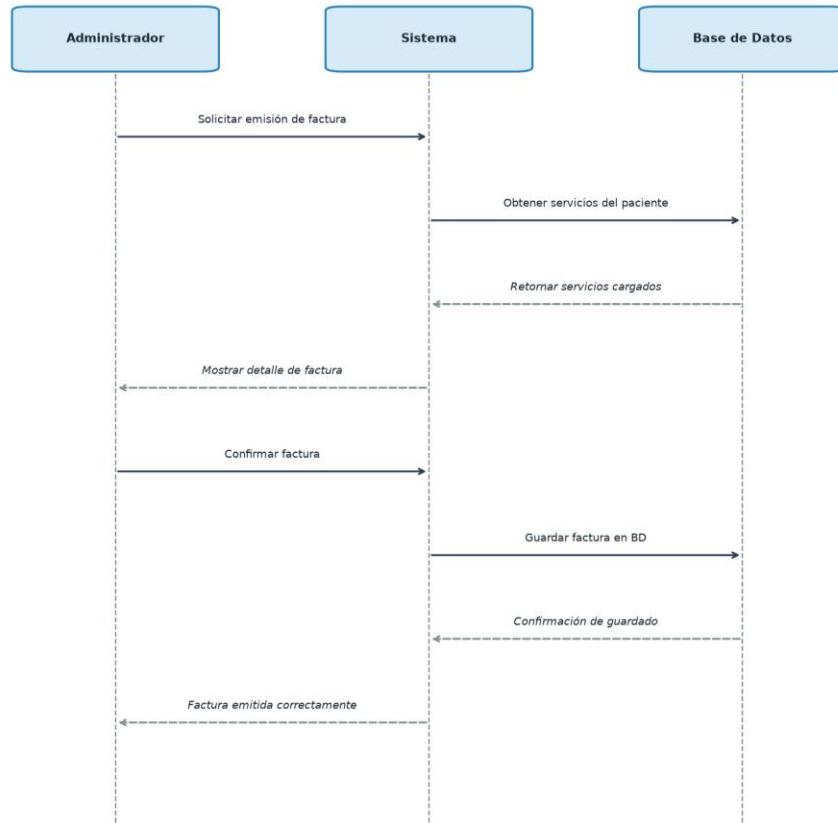
Este diagrama ilustra el proceso mediante el cual un médico accede y actualiza la historia clínica de un paciente. El sistema valida las credenciales del médico, recupera la información existente del paciente y persiste el nuevo diagnóstico o tratamiento en la base de datos.



3.3 Facturación

El diagrama de facturación describe cómo el administrador genera una factura por los servicios prestados. El sistema obtiene los servicios del paciente, muestra el detalle para su confirmación y persiste la factura en la base de datos con su estado correspondiente.

Diagrama de Secuencia: Facturación



4. Etapa de Implementación

4.1 Desarrollo del Sistema

El sistema fue desarrollado íntegramente en Java (JDK 21), organizando el código en módulos independientes por funcionalidad. Se implementó el patrón de diseño DAO (Data Access Object), que permite separar completamente la lógica de negocio del acceso a la base de datos. Esto facilita el mantenimiento del código, ya que cualquier cambio en la capa de datos no impacta en la lógica de la aplicación.

Cada entidad del sistema (Paciente, Médico, Cita, Producto, Factura) cuenta con su propia clase DAO que implementa las operaciones CRUD completas: insertar, obtener por ID, obtener todos, actualizar y eliminar.

4.2 Conexión con la Base de Datos

La conexión a MySQL se realiza a través del driver JDBC (mysql-connector-java). La clase ConexionDB centraliza la configuración de la conexión, evitando duplicación de parámetros en el código:

```
public class ConexionDB {
    private static final String URL =

    "jdbc:mysql://localhost:3306/clinicadb?useSSL=false&serverTimezone=UTC";
    private static final String USER = "root";
    private static final String PASSWORD = "123456";

    public static Connection getConexion() throws SQLException {
        return DriverManager.getConnection(URL, USER, PASSWORD);
    }
}
```

Esta clase es utilizada por todos los DAOs a través de AbstractDAO, que establece la conexión en su constructor y la comparte con las subclases.

4.3 Patrón DAO

La interfaz DAO<T> define el contrato que deben cumplir todos los DAOs del sistema. Esto garantiza coherencia en las operaciones disponibles para cada entidad:

```
public interface DAO<T> {
    void insertar(T t) throws SQLException;
    T obtener(int id) throws SQLException;
    List<T> obtenerTodos() throws SQLException;
    void actualizar(T t) throws SQLException;
    void eliminar(int id) throws SQLException;
}
```

Cada DAO concreto (PacienteDAO, MedicoDAO, etc.) extiende AbstractDAO<T> e implementa estas operaciones usando PreparedStatement para prevenir inyección SQL y mejorar el rendimiento en consultas repetidas.

4.4 Repositorio del Proyecto

El código fuente completo del proyecto se encuentra disponible en:

<https://github.com/antonellaanahi/SEMINARIODEPRACTICA>

5. Etapa de Pruebas

Para validar el correcto funcionamiento del sistema se llevaron a cabo distintos niveles de prueba, asegurando tanto la corrección individual de cada componente como la integración entre módulos.

Pruebas Unitarias

Se verificó que cada método de los DAOs funcione correctamente de forma independiente. Por ejemplo, se comprobó que insertar un paciente genera el registro en la base de datos con todos sus atributos correctamente asignados, y que obtener por ID retorna el objeto esperado con los datos persistidos.

Pruebas de Integración

Se validó la interacción entre módulos. En particular, se verificó que al crear una cita los IDs de paciente y médico referenciados existan previamente (clave foránea), y que el módulo de reportes integre correctamente los datos de pacientes, citas e inventarios.

Pruebas de Sistema

Se ejecutaron los flujos completos del sistema: desde el registro de un paciente, pasando por la programación de un turno y el registro de una historia clínica, hasta la emisión de la factura correspondiente. Se verificó que los datos persistan correctamente en cada etapa.

Pruebas de Usuario

Se simularon escenarios de uso real con el menú de consola, incluyendo casos de error como intentar asignar una cita sin pacientes registrados o ingresar un ID inexistente. Se validó que el sistema responda con mensajes claros y no se produzcan excepciones no controladas.

6. Definición de Base de Datos para el Sistema

La base de datos ClinicaDB fue diseñada aplicando las tres primeras formas normales de la normalización relacional, con el objetivo de eliminar redundancias y garantizar la integridad de la información.

Primera Forma Normal (1FN)

Se verificó que cada tabla contenga únicamente atributos atómicos (sin grupos repetitivos ni valores múltiples en un mismo campo). Cada tabla posee una clave primaria única de tipo AUTO_INCREMENT que identifica de forma inequívoca cada registro.

Segunda Forma Normal (2FN)

Todos los atributos de cada tabla dependen funcionalmente de la clave primaria completa. La información de pacientes, médicos, proveedores y facturas se almacena en tablas separadas, evitando que un mismo dato aparezca duplicado en varias tablas. Por ejemplo, los datos del paciente no se repiten en la tabla Citas sino que se referencian mediante la clave foránea paciente_id.

Tercera Forma Normal (3FN)

Se eliminaron las dependencias transitivas. La información de proveedores, por ejemplo, no se almacena dentro de la tabla Inventarios sino en su propia tabla Proveedores, relacionada mediante proveedor_id. De este modo, cualquier cambio en los datos de un proveedor se realiza en un único lugar.

7. Diagrama Entidad-Relación de la Base de Datos

El siguiente diagrama muestra las entidades del sistema, sus atributos y las relaciones entre ellas. Las claves primarias se indican con "PK" y las claves foráneas con "FK". Las relaciones son de tipo uno-a-muchos (1:N) en todos los casos.



Relaciones principales: Pacientes y Médicos tienen relación 1:N con Citas e Historias_Clinicas. Pacientes tiene relación 1:N con Facturas. Proveedores tiene relación 1:N con Inventarios. Reportes es una entidad independiente.

8. Creación de las Tablas MySQL

A continuación se presentan los scripts SQL para la creación de la base de datos y sus tablas. Las claves foráneas aseguran la integridad referencial entre entidades. La tabla Proveedores se crea antes que Inventarios dado que esta última la referencia.

Base de datos

```
CREATE DATABASE IF NOT EXISTS ClinicaDB;
USE ClinicaDB;
```

Tabla Pacientes

```
CREATE TABLE Pacientes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    fecha_nacimiento DATE,
    direccion VARCHAR(100),
    telefono VARCHAR(20),
    correo_electronico VARCHAR(50)
);
```

Tabla Médicos

```
CREATE TABLE Medicos (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(50) NOT NULL,
    apellido VARCHAR(50) NOT NULL,
    especialidad VARCHAR(50),
    telefono VARCHAR(20),
    correo_electronico VARCHAR(50)
);
```

Tabla Proveedores

```
CREATE TABLE Proveedores (
    id INT AUTO_INCREMENT PRIMARY KEY,
    nombre VARCHAR(100) NOT NULL,
    contacto VARCHAR(100)
);
```

Tabla Citas

```
CREATE TABLE Citas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT NOT NULL,
    medico_id INT NOT NULL,
    fecha_hora DATETIME NOT NULL,
    estado VARCHAR(20) DEFAULT "Pendiente",
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);
```

Tabla Historias_Clinicas

```
CREATE TABLE Historias_Clinicas (
    id INT AUTO_INCREMENT PRIMARY KEY,
```

```

    paciente_id INT NOT NULL,
    medico_id INT NOT NULL,
    fecha DATE NOT NULL,
    detalles TEXT,
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id),
    FOREIGN KEY (medico_id) REFERENCES Medicos(id)
);

```

Tabla Inventarios

```

CREATE TABLE Inventarios (
    id INT AUTO_INCREMENT PRIMARY KEY,
    producto VARCHAR(100) NOT NULL,
    cantidad INT DEFAULT 0,
    fecha_vencimiento DATE,
    proveedor_id INT,
    FOREIGN KEY (proveedor_id) REFERENCES Proveedores(id)
);

```

Tabla Facturas

```

CREATE TABLE Facturas (
    id INT AUTO_INCREMENT PRIMARY KEY,
    paciente_id INT NOT NULL,
    fecha DATE NOT NULL,
    total DECIMAL(10,2),
    estado VARCHAR(20) DEFAULT "Pendiente",
    FOREIGN KEY (paciente_id) REFERENCES Pacientes(id)
);

```

Tabla Reportes

```

CREATE TABLE Reportes (
    id INT AUTO_INCREMENT PRIMARY KEY,
    tipo_reporte VARCHAR(50),
    fecha_creacion DATE,
    detalles TEXT
);

```

9. Inserción, Consulta y Borrado de Registros

Las siguientes sentencias SQL muestran las operaciones básicas de manipulación de datos (DML) sobre las tablas principales del sistema.

Inserción de registros

Se insertan registros de ejemplo para pacientes y médicos:

```
INSERT INTO Pacientes (nombre, apellido, fecha_nacimiento, direccion,
telefono, correo_electronico)
VALUES ('Juan', 'Pérez', '1985-03-15', 'Calle Falsa 123', '123456789',
'juan.perez@email.com');
```

```
INSERT INTO Pacientes (nombre, apellido, fecha_nacimiento, direccion,
telefono, correo_electronico)
VALUES ('Maria', 'López', '1990-07-22', 'Av. Siempreviva 742',
'987654321', 'maria.lopez@email.com');
```

```
INSERT INTO Medicos (nombre, apellido, especialidad, telefono,
correo_electronico)
VALUES ('Carlos', 'Gómez', 'Cardiología', '111222333',
'cgomez@clinica.com');
```

```
INSERT INTO Citas (paciente_id, medico_id, fecha_hora, estado)
VALUES (1, 1, '2025-07-10 09:00:00', 'Pendiente');
```

Consulta de registros

Ejemplo de consulta de todos los pacientes registrados:

```
SELECT * FROM Pacientes WHERE apellido = 'Pérez';
```

Borrado de registros

Eliminación de un registro por su clave primaria:

```
DELETE FROM Pacientes WHERE id = 1;
```


10. Presentación de Consultas SQL

A continuación se presentan consultas SQL representativas del sistema, organizadas por funcionalidad:

Listar citas por médico

Obtiene todas las citas asignadas a un médico específico:

```
SELECT c.id, CONCAT(p.nombre, " ", p.apellido) AS paciente,
       c.fecha_hora, c.estado
FROM Citas c
JOIN Pacientes p ON c.paciente_id = p.id
WHERE c.medico_id = 1;
```

Buscar pacientes por nombre

Búsqueda parcial de pacientes usando LIKE:

```
SELECT * FROM Pacientes WHERE nombre LIKE '%Juan%';
```

Inventario próximo a vencer

Productos cuya fecha de vencimiento es en los próximos 30 días:

```
SELECT * FROM Inventarios
WHERE fecha_vencimiento < CURDATE() + INTERVAL 30 DAY
ORDER BY fecha_vencimiento ASC;
```

Total de ingresos por facturas pagadas

Suma de los montos de todas las facturas con estado "Pagada":

```
SELECT SUM(total) AS ingresos_totales
FROM Facturas WHERE estado = 'Pagada';
```

Historial clínico de un paciente

Todas las consultas médicas registradas para un paciente:

```
SELECT h.fecha, CONCAT(m.nombre, " ", m.apellido) AS medico, h.detalles
FROM Historias_Clinicas h
JOIN Medicos m ON h.medico_id = m.id
WHERE h.paciente_id = 1
ORDER BY h.fecha DESC;
```

11. Definiciones de Comunicación

Esta sección define los aspectos relacionados con la comunicación entre los componentes del sistema y con el entorno de red en el que operará.

11.1 Entorno de Red

El sistema utiliza el protocolo TCP/IP para la comunicación entre la aplicación Java y el servidor MySQL. En la configuración actual, tanto la aplicación como la base de datos se ejecutan en el mismo equipo (localhost, puerto 3306), lo que elimina latencia de red en el entorno de desarrollo. En un entorno productivo, la base de datos podría ubicarse en un servidor dedicado, siendo necesario configurar reglas de firewall y acceso remoto en MySQL para permitir la conexión.

11.2 Seguridad en la Transmisión

La conexión JDBC se configura con el parámetro useSSL=false para el entorno de desarrollo local. En un entorno de producción, se recomienda habilitar SSL/TLS para cifrar la transmisión de datos entre la aplicación y el servidor MySQL, protegiendo información sensible como datos de pacientes y registros médicos.

11.3 Integridad y Confidencialidad

La integridad de los datos se garantiza mediante el uso de transacciones y restricciones de clave foránea a nivel de base de datos. El acceso a la información está controlado por el sistema de autenticación de MySQL (usuario y contraseña). Para un entorno productivo, se recomienda implementar roles de usuario en la base de datos, limitando los permisos de cada perfil (lectura, escritura, administración) según su función.

11.4 Interoperabilidad

El sistema fue desarrollado con Java, un lenguaje multiplataforma, lo que permite su ejecución en distintos sistemas operativos sin modificaciones. La base de datos MySQL es ampliamente compatible con otros sistemas y herramientas de análisis. En futuras versiones, se podría exponer una API RESTful para facilitar la integración con sistemas externos, como laboratorios, obras sociales o plataformas de teleasistencia.